

Jengine Format

Jengine Format

What is jengine

How jengine works

- Overall request design

- Overall response design

 - First Stage

 - Second stage

 - code

 - error

Action list

- Overview

- 00 generic

 - set_req

 - set_rsp

- 02 image

 - set_req

 - set_rsp

- 03 ota

 - set_req

 - set_rsp

- 20 query

 - set_req

 - set_rsp

- 26 shutdown

 - set_req

 - set_rsp

- 33 refresh

 - set_req

 - set_rsp

- 36 slot

 - field decriptions

 - common

 - frame_type

 - special for ibeacon

 - special for uid

 - special for url

 - special for string

 - get_req

 - get_rsp

 - set_req

 - set_rsp

- 37 led via radio

 - set_req

 - set_rsp

- 42 led via BLE

 - set_req

 - set_rsp

 - color-table

- 44 factoryreset

 - set_req

 - set_rsp

- 52 trigger

	trigger_source
	get_req
	get_rsp
	set_req
	set_rsp
54	HT data
	get_req
	get_rsp
56	buzzer
	set_req
	set_rsp
57	scan
	get_req
	get_rsp
	set_req
	set_rsp
58	reboot
	set_req
	set_rsp
62	timestamp
	get_req
	get_rsp
	set_req
	set_rsp
63	historical data
	rule
	type
	get_req
	get_rsp
67	switches
	get_req
	get_rsp
	set_req
	set_rsp
	option
74	version
	get_req
	get_rsp
	firmware_type
75	operating status
	get_req
	get_rsp
101	Multi-Screen Image Region Refresh via Radio
	set_req
	set_rsp
102	Multi-Screen Image Region Refresh via BLE
	set_req
	set_rsp
103	Multi-Panel Image Download & Screen Refresh
	set_req
	set_rsp

FAQ

- Q1
- Q2
- Q3
- Q4

WiFi Version	Released	Remark
v3.4.0	2023-05-05	1. First draft
v3.5.0	2023-10-18	1. D2C, Direct Method, Device Twins features @ Azure IoT Hub 2. shadow service feature @ AWS IoT Core 3. Support for sending commands to standard-version devices(private 2.4GHz radio-based devices) via Jengine.
v3.5.0	2023-11-22	1. Support stage 2 result returned at private 2.4GHz radio
v3.5.0	2023-11-23	1. Support to use suppress_stage2 option to suppress stage 2 result if you don't need it. 2. Add more detailed FAQ info. 3. Add more detailed info about 02 image
v3.5.0	2023-12-19	1. Correct rssi_at_xm range to -127 ~ 0 2. Correct generic.params.scan_number range to 1 ~ 3
v3.5.0	2024-01-22	1. Note that 1234567890123456 is not the actual device key 2. Note that do not encode image/ota/.. data in HTTP[s] link
v3.5.1	2024-03-13	1. Added commands 54, 62, 63
v3.5.1	2024-03-18	1. Add the description of error code 710
v3.5.1	2024-03-25	1. Rename the commands 54 and 63, enabling users to effectively distinguish between them.
v3.5.1	2024-04-02	1. Add more detailed info about command 54.
v3.5.1	2024-04-11	1. Added pre-stored image function to jengine command 02; 2. Added jengine command 33 to trigger specified image area refresh function.
v3.7.0	2025-04-15	1. Maintain jengine format in a separate document.
v3.7.0	2025-04-15	1. To clarify, the previously mentioned standalone version (or single version) of the tag device will be named the BLE Version , while the standard version will be named the MIX Version .

WiFi Version	Released	Remark
v3.7.0	2025-04-25	1. Added commands 101 (Multi-Screen Image Region Refresh via Radio), 102 (Multi-Screen Image Region Refresh via BLE) and 103 (Five-Panel Image Download & Screen Refresh).
v3.4.0	2025-04-25	1. Added description for command 0, which has been available since the initial v3.4.0 release but is now properly documented; Command 0 is designed for emergency use, although it requires additional hexadecimal MDU private format, making it slightly more complex to use; In fact, all other BLE channel commands can be implemented through command 0.

This document's functionality is only applicable to the following firmware versions and models.

Firmware Model	Minimum Supported Wi-Fi Firmware Version	Notes
g1-d-pro-advanced	v3.4.0	G1-D Gateway
g1-e-grapes	v3.4.0	G1-E Gateway
mp01h-plum	v3.5.0	Huawei PCIe IoT Card Firmware
mg6-a-kiwi	v3.6.0	MG6-A Gateway

What is jengine

Due to the fact that the format of `Minew-Connect V3 protocol` belongs to hexadecimal format, it is difficult for users to understand. The purpose of this jengine is to encapsulate a layer of JSON format for the platform, which is responsible for converting the JSON format issued by the platform into Minew-Connect V3 format, or converting the returned Minew-Connect V3 format data into JSON format.

How jengine works

- Jengine feature was introduced in wifi version `v3.4.0`.
- The gateway will subscribe to the `action topic` (default to `/gw/{mac}/action`). Once the `backend` sends a command to the corresponding gateway's action topic, the gateway would execute the command.
- After executing the command based on the content received from the `action topic`, gateway will publish the result in the `response topic` (default to `/gw/{mac}/response`). The `backend` needs to subscribe to the `response topic` to obtain the result of the command.
- The addition of the `Broker Type` configuration parameter was introduced in wifi version `v3.5.0`.

- If you select the **Azure IoT Hub** type, as shown in the diagram below, the **jengine** requests to Azure IoT Hub should utilize [Direct Method](#).

Direct method ...

ac233fc03ce9

You can use this tool to invoke direct methods on devices from the cloud. Direct methods have a name, payload, and configurable timeouts. [Learn more](#)

Device ID

ac233fc0



Method name * ⓘ

jengine

Payload ⓘ

```
{
  "action": 42,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456",
    "color": 3,
    "cycles": 20,
    "light_on": 300,
    "light_off": 300,
    "brightness": 50,
    "details": {
      "ac1122334455": {}
    }
  }
}
```

Response timeout ⓘ

30 seconds

Connection timeout ⓘ

Device must already be connected

Invoke method

Result

```
{
  "status": 200,
  "payload": {
    "action": 42,
    "req_id": 123,
    "version": 1,
    "method": "set_rsp",
    "payload": {
      "code": 1,
      "message": "Success to receive the command, please wait."
    }
  }
}
```

- The **Method name** parameter should be set as **jengine**, and the **Payload** parameter should be filled with the desired JSON content for the request. The invoked result will directly return the results of the first stage, which is included in **'status': <http-type-code>**, **'payload': <first-stage-result>**.
- The results of the second stage are reported through the designated topic specified by [device-to-cloud](#), which topic is **devices/{device-id}/messages/events/event_type=res&%24.ct=application%2fjson&%24.ce=utf-8**, labeled as **event_type=res** property.

Overall request design

Here is the description of the jengine fields:

Field	Type	Required	Value	Remark
action	number	Yes	Each command has a specific value, given in the specific command	Used to identify and distinguish commands
version	number	Yes	1 or 2	The current version is 1 or 2, used to identify the JSON request data structure version
method	string	Yes	<code>get_req</code> <code>set_req</code>	1. <code>get_req</code> indicates that the command type sent is a parameter request (get method); 2. <code>set_req</code> indicates that the command type sent is for configuration or control of the device (set method)
req_id	number	No	A 4-byte positive integer, returning failure if out of range, req_id cannot be 0	Since command requests and results are not returned in real time, req_id is used to match requests and returns
cmds	array	Included in the v2 version only	array	1.Content used to identify multiple commands; 2.This field is only available for v2 version, which was introduced after v3.7.0;
payload	object	Yes	object	Stores the common part parameters of the request or the returned result

Field	Type	Required	Value	Remark
payload.key	string	Yes	16-digit password to connect to the device	Password for gateway to connect to the device (public part)
payload. {others}	number string boolean array	/	/	Other parameters required by various commands (public part)
payload.details	object	Yes	An object with the <code>key</code> being the MAC address of the device end	Store parameters owned by each individual device.
payload.details. {mac}	object	Yes	object	1.Stores the parameters for this MAC address device, used to completely override the parameters in payload, see the description of Request Example 1 2.Note that the {mac} address must be lowercase and cannot have any other separators, otherwise the gateway will return a format error
payload.details. {mac}.key	string	When payload.key does not exist, it is required	16-digit password to connect to the device	1. Password for gateway to connect to the device, only applicable to devices with this {mac} address; 2. If payload.key also exists, it will override the value of payload.key, and the value of payload.details. {mac}.key will be used first

Field	Type	Required	Value	Remark
payload.details. {mac}.{others}	number string boolean array	/	/	1. Other parameters required by various commands. 2. If payload.{others} also exists, it will override the value of payload.{others}, and the value of payload.details.{mac}.{others} will be used first

The following three request examples are all correct.

- **Request Example 1**

```
{
  "action": 0,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "a234567890123456",
    "others": "bar",
    "details": {
      "ac1122334455": {
        "key": "b234567890123456",
        "others": "foo"
      },
      "ac1122334456": {},
      "ac1122334457": {
        "key": "c234567890123456"
      },
      "ac1122334458": {}
    }
  }
}
```

Q: Why are there two identical parameters such as payload.key and payload.details.{mac}.key, payload.others and payload.details.{mac}.others? What is the difference between these two parameters?

A: When the parameter exists in payload.details.{mac}, the gateway will prioritize using the parameter from payload.details.{mac}. If there is no parameter, the device with that MAC address will directly use the parameter from payload. Therefore, when making a request like in example 1, the device parameters used in the request are as follows:

mac	key	others
ac1122334455	b234567890123456	foo

mac	key	others
ac1122334456	a234567890123456	bar
ac1122334457	c234567890123456	bar
ac1122334458	a234567890123456	bar

The advantage of designing it this way is that when all parameters are the same, common parameters can be placed in payload.{others}, which saves the JSON size of batch requests.

- **Request Example 2**

```
{
  "action": 0,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "a234567890123456",
    "others": "bar",
    "details": {
      "ac1122334455": {},
      "ac1122334456": {},
      "ac1122334457": {},
      "ac1122334458": {}
    }
  }
}
```

- **Request Example 3**

```
{
  "action": 0,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "key": "b234567890123456",
        "others": "foo"
      },
      "ac1122334456": {
        "key": "a234567890123456",
        "others": "bar",
      },
      "ac1122334457": {
        "key": "c234567890123456",
        "others": "bar"
      },
      "ac1122334458": {
        "key": "a234567890123456",
        "others": "bar"
      }
    }
  }
}
```

```
}  
}
```

- **Request Example 4**

The request style for `jengine v2` is as shown below. The examples in the later part of this document are based on `jengine v1`. If you have requirements for using `jengine v2`, you can refer to this style for rewriting. For more detailed content, please refer to [Q5](#).

```
{  
  "version": 2,  
  "req_id": 123,  
  "cmds": [  
    {  
      "action": 56,  
      "method": "set_req",  
      "payload": {  
        "key": "3141592653589793",  
        "opcode": 22345,  
        "cycles": 2,  
        "work": 30,  
        "idle": 300,  
        "level": 0,  
        "details": {  
          "c3000013896f": {},  
          "c30000138970": {}  
        }  
      }  
    },  
    {  
      "action": 42,  
      "method": "set_req",  
      "payload": {  
        "key": "3141592653589793",  
        "opcode": 22345,  
        "single": true,  
        "color": 3,  
        "cycles": 20,  
        "light_on": 300,  
        "light_off": 300,  
        "brightness": 50,  
        "details": {  
          "c3000013896f": {},  
          "c30000138970": {}  
        }  
      }  
    },  
    {  
      "action": 74,  
      "method": "get_req",  
      "payload": {  
        "key": "3141592653589793",  
        "opcode": 22345,  
        "details": {  
          "c30000138970": {}  
        }  
      }  
    }  
  ]  
}
```

```

    }
  }
]
}

```

`jengine v2` was introduced in v3.7.0 and added the ability to issue multiple commands in a one message, a one message can contain multiple different action fields. For example, you can send `42 led via BLE` and `56 buzzer` at the same time, no longer need to send two commands via two messages.

Note that the `radio`, `single`, and `opcode` fields must be the same in a one message. The single command(BLE version) and the standard command(MIX version) cannot be issued together

Overall response design

- The gateway returns the processing result in two stages:
 - In the first stage, the format of the JSON issued is checked and the parameters are pre-validated. If an error is returned in the first stage, there will be no second stage return.
 - The second stage is the return of the execution after the BLE connection, which is the result of attempting to connect to each device.

Field	Type	Included	Value	Remark
action	number	YES	Each command has a specific value, defined within the command itself.	1. Used to identify the command; 2. When there is an error in the request format, the returned value for the action may not equal the value of the issued action.
version	number	YES	1 or 2	The current version is 1 or 2, used to identify the JSON request data structure version
method	string	YES	get_rsp set_rsp	1. <code>get_rsp</code> indicates the return result of the get method ; 2. <code>set_rsp</code> indicates the return result of the set method

Field	Type	Included	Value	Remark
req_id	number	Included if the request also contains	The returned value equals the issued value	1.If there is an issue with the format of the issued command, the returned value may not necessarily equal the issued value; 2.If this field is absent from the command, it will not appear in the response.
cmds	array	Included in the v2 version only	array	1.Content used to identify multiple commands; 2.This field is only available for v2 version, which was introduced after v3.7.0;
payload	object	YES	object	
payload.code	number	Included in the first stage only	code	1. The gateway returns the error code result processed in the first stage. 2. If there are no errors in the first stage, it returns 1.
payload.message	string	Included in the first stage only	/	Here is some descriptive information about the <code>payload.code</code> .
payload.details	object	Included in the second stage only	/	Store parameters owned by each individual device.
payload.details.{mac}	object	Included in the second stage only	object	Store the return information for the device with this MAC address.

Field	Type	Included	Value	Remark
payload.details. {mac}.code	number	Included in the second stage only	code	1. This represents the error code result of the second-stage processing. 2. When this value is 2, it is necessary to combine the error code <code>payload.details.{mac}.error</code> to determine whether the command was successfully executed on the device side. 3. If the value returned by the <code>code</code> in the second stage is not 2, there is definitely an error, and checking the <code>error</code> field is unnecessary at this point.
payload.details. {mac}.message	string	Included in the second stage only	/	Here is some descriptive information about the <code>payload.details.{mac}.code</code>
payload.details. {mac}.error	number	Included in the second stage only	error	1. This value is the error code returned by the device after processing a command. 2. When this value is not equal to 0, it indicates that there was an error in the processing.
payload.details. {mac}.{others}	number string boolean array	Depends on various actions	/	This is other information obtained from the device in the second stage, and the specific details depend on the definition of each command.

First Stage

- **Response Example 1**

```
{
  "action": 0,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "code": 1, /* Success receive the command */
    "message": "Success to receive the command, please wait!"
  }
}
```

- **Response Example 2**

```
{
  "action": 0,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "code": 100, /* Format error */
    "message": "xxxx"
  }
}
```

- **Response Example 3**

jengine v2

```
{
  "version" : 2,
  "req_id" : 123,
  "cmds" : [ {
    "action" : 56,
    "method" : "set_rsp",
    "payload" : {
      "code" : 0,
      "message" : "Command parsed successfully."
    }
  }, {
    "action" : 42,
    "method" : "set_rsp",
    "payload" : {
      "code" : 0,
      "message" : "Command parsed successfully."
    }
  }, {
    "action" : 74,
    "method" : "get_rsp",
    "payload" : {
      "code" : 300, /* Format error */
      "message" : "The `single` or `opcode` parameter cannot vary within a single request!"
    }
  }
}
```

```
} ]  
}
```

- **Response Example 4**

jengine v2

```
{  
  "version" : 2,  
  "req_id" : 123,  
  "cmds" : [ {  
    "action" : 56,  
    "method" : "set_rsp",  
    "payload" : {  
      "code" : 1,  
      "message" : "Success to receive the command, please wait."  
    }  
  }, {  
    "action" : 42,  
    "method" : "set_rsp",  
    "payload" : {  
      "code" : 1,  
      "message" : "Success to receive the command, please wait."  
    }  
  }, {  
    "action" : 74,  
    "method" : "get_rsp",  
    "payload" : {  
      "code" : 1,  
      "message" : "Success to receive the command, please wait."  
    }  
  }  
] ]  
}
```

Second stage

- **Response Example 1**

```
/* Each device returns results separately */  
{  
  "action": 0,  
  "version": 1,  
  "method": "get_rsp",  
  "req_id": 123,  
  "payload": {  
    "details": {  
      "ac1122334455": {  
        "code": 2, /* Successful transmission */  
        "error": 0, /* Device side execution successful */  
        "message": "Success!",  
        "others": "bar" /* Return the parameters that have been fetched  
*/  
      }  
    }  
  }  
}
```

```

}

{
  "action": 0,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334456": {
        "code": 2, /* Successful transmission */
        "error": 20, /* But the tag device side claims that your
parameters are in error */
        "message": "Success to transmit command data to device!"
      }
    }
  }
}

// ...

```

- **Response Example 2**

jengine v2

```

/* Each device returns results separately */
{
  "version" : 2,
  "req_id" : 123,
  "cmds" : [ {
    "action" : 56,
    "method" : "set_rsp",
    "payload" : {
      "details" : {
        "c30000138970" : {
          "code" : 2,
          "error" : 0,
          "message" : "Success to transmit command data to device!"
        }
      }
    }
  }, {
    "action" : 42,
    "method" : "set_rsp",
    "payload" : {
      "details" : {
        "c30000138970" : {
          "code" : 2,
          "error" : 0,
          "message" : "Success to transmit command data to device!"
        }
      }
    }
  }, {
    "action" : 74,

```

```

    "method" : "get_rsp",
    "payload" : {
      "details" : {
        "c30000138970" : {
          "code" : 2,
          "error" : 0,
          "message" : "Success to transmit command data to device!",
          "firmware" : [ {
            "type" : 0,
            "version" : "v3.2.0"
          } ]
        }
      }
    }
  }
} ]
}

```

```

{
  "version" : 2,
  "req_id" : 123,
  "cmds" : [ {
    "action" : 56,
    "method" : "set_rsp",
    "payload" : {
      "details" : {
        "c3000013896f" : {
          "code" : 2,
          "error" : 0,
          "message" : "Success to transmit command data to device!"
        }
      }
    }
  }, {
    "action" : 42,
    "method" : "set_rsp",
    "payload" : {
      "details" : {
        "c3000013896f" : {
          "code" : 2,
          "error" : 0,
          "message" : "Success to transmit command data to device!"
        }
      }
    }
  }
]
}

```

```
// ...
```

code

- Applied to **payload.code** or **payload.details.{mac}.code**;
- Error codes between 0 and 99 indicate that the execution was successful, while values greater than or equal to 100 indicate that an error has occurred.
- Only some commonly used error codes are listed here. If the error codes you encountered are not in this table, you can consult us.
- Most errors may be caused by factors such as distance, resulting in unsuccessful connections. You can try resending the command to the closer gateway see if it works.
- The error codes here represent the results of execution on the gateway side and do not represent the final results. The execution results on the device side need to be checked in the [error list](#).

Code	Stage	Remark
1	1	The gateway has successfully received the JSON command you requested, and the parameters have passed the verification. It is queuing to process the ble connection.
2	2	(BLE connection command)The gateway has successfully delivered the command you want to send to the device side. At this point, it is impossible to determine whether the command has been successfully executed on the device side based solely on this 2 error code. Please refer to the error section for more information.
3	2	(2.4GHz radio command)The gateway has successfully notify the device. Due to the high success rate typically associated with the 2.4GHz radio method, you can consider the device operation successful based on this error code 3. If you require a 100% success rate, you can use both the opcode and the error code for determination. However, except for the 20 query command, since this command is intended for querying the state of the device, the device will not change the opcode value after receiving the 2.4GHz radio command. Therefore, you only need to check for error code 3.
100	1	The format issued by the platform is incorrect.
200	1	Internal gateway error, usually due to frequent requests and insufficient memory.
300	1	Parameter verification error.
400	1	Network error, indicating that when there are HTTP links in the parameters, the link cannot be downloaded.
677	2	Connection authentication error, possibly due to incorrect password.

Code	Stage	Remark
701	2	Failed to wake up the device via 2.4GHz radio command! Possible reasons include: 1. The device is not within the range of the gateway, and you need to send the command to the gateway closest to the BLE device; 2. The MAC address of the BLE device is incorrect; 3. The BLE device is out of power or is malfunctioning.
708	2	The gateway cannot scan the advertising packet of the BLE device, so it has not initiated a BLE connection. Possible reasons include: 1. The device is not within the range of the gateway, and you need to send the command to the gateway closest to the BLE device; 2. The MAC address of the BLE device is incorrect; 3. The BLE device is out of power or is malfunctioning.
710	2	Several scenarios could lead to this error: 1. When multiple gateways are deployed in the environment, it indicates that device may have been successfully connected to other gateways. 2. It could also be an issue with the opcode parameter. Ensure that each time a command is issued, your opcode value differs from the previous one.

error

- Applied to **payload.details.{mac}.error** or the **error** field in `Json-Prepared`'s `ds` type packet.
- Any value other than **0** for the error code indicates an error.
- Only some commonly used error codes are listed here. If the error codes you encountered are not in this table, you can consult us.

Error	Remark
0	No error
4	Connection authentication error
13	This error code indicates an invalid BLE interaction command, which means that the device or firmware version does not support the command yet.
20	This error code indicates a parameter error carried when issuing the command.
26	The size of the image sent does not match the device
41	Incorrect compressed data format
44	A-side screen refresh unsuccessful
45	B-side screen refresh unsuccessful
46	Both A & B-side screen refresh unsuccessful

Action list

Overview

- Support for standard-version(private 2.4GHz radio-based) device since `v3.5.0`.
- BLE devices have two versions: single version and standard version. What are the differences between these two versions?
 - The single version, also known as the long advertising version, is a device that continuously broadcasts and does not enter sleep mode. If you need the device to be connectable by a smartphone, the single version is generally preferred.
 - The standard version, also known as the gateway version, is a device that doesn't typically broadcast advertisements. When a connection to the device is required, a gateway uses its private 2.4GHz radio to wake up the device and establish a connection. Since BLE devices in the standard version don't broadcast frequently, they cause less interference and are suitable for large-scale deployments. Mobile phones cannot utilize the 2.4GHz radio for private purposes, so they cannot directly connect to and control standard version devices.
- How can we distinguish between the two types(single or standard) of BLE devices?
 - The type of device depends on the firmware used by the device. Generally, this is predetermined and fixed during the manufacturing process. You can also directly consult with us to determine the device type.
 - When a BLE device has a "ds"(dynamic status) advertising packet, you can differentiate them based on the "single" field in the "ds" advertising packet(Json-Prepared format).
- It should be noted that the standalone version (single version) of the tag device will be named the `BLE Version`, while the previously mentioned standard version will be named the `MIX Version`. In other words:
 - `BLE Version` = `Single Version`
 - `MIX Version` = `Standard Version`
- Common parameters for some actions are list here.
 - The following common parameter should be placed at payload key or payload.details.{mac}.

Field	Type	Required	Value	Remark
key	string	Required if the action connected with BLE	16-digit password to connect to the device	1. Password for gateway to connect to the device; 2.It is not required if the action is based on a private 2.4GHz radio.

Field	Type	Required	Value	Remark
opcode	number	/	1~UINT32_MAX	Use a new opcode every time you issue a command to match requests and results. This <code>opcode</code> will be returned in the adv packet(See <code>ds</code> packet in Json-Prepared format for more details)
single	boolean	No	true/false Defaults to true if this field is not provided	Introduced since v3.5.0 <code>True</code> means a single-version device(BLE version), while <code>false</code> means a standard-version device(MIX version)(private 2.4GHz radio-based).
suppress_stage2	boolean	No	true/false Defaults to false if this field is not provided	Introduced since v3.5.0 <code>True</code> means suppress the stage 2 result and don't return the stage 2 result, while <code>false</code> means return the stage 2 result.

- The commands listed below are for various Minew BLE devices. Your BLE device may not support all commands.

00 generic

- This is a generic command for data pass-through to devices.
- It can be used as an emergency or backup solution when the gateway hasn't adapted to a specific command (limited to commands with simple logic).
- This command allows you to transparently transmit MDU command data to the device.

Field	Type	Required	Value	Remark
mdu	string	Yes	Base64-encoded string of MDU command data	The MDU command data format is complex and requires additional documentation. The example given here is the broadcast interval for configuring BLE version(Single version) tag devices.

set_req

```
{
  "action": 0,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "3141592653589793",
    "single": true,
    "mdu": "<The mdu command data to device>",
    "details": {
      "<ble device mac address>": {}
    }
  }
}
```

For example, to set the broadcast interval of a BLE version device with MAC address ac233fd00c31 to 1800ms, you can issue the following command:

```
{
  "action": 0,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "3141592653589793",
    "single": true,
    "mdu": "EQABAAAAA==",
    "details": {
      "ac233fd00c31": {}
    }
  }
}
```

The Base64-encoded MDU data corresponds to different broadcast intervals:

- 900ms: EQABAAAAAQ==
- 1800ms: EQABAAAAA==
- 2000ms: EQABAAAAAg==
- 3000ms: EQABAAAAAw==
- 4000ms: EQABAAAABA==

set_rsp

```
{
  "action": 0,
  "req_id": 123,
  "version": 1,
  "method": "set_rsp",
  "payload": {
    "details": {
      "ac233fd00c31": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!"
      }
    }
  }
}
```

02 image

- Support `set` method only.
- Used to transmit image data to devices.
- Since v3.5.1, the `images.region` and `images.refresh` parameters have been added to manage pre-stored images. If your device is a single version, please ignore this, as currently only the `33 refresh` command for private 2.4GHz radio-based devices can trigger image refresh. There is currently no command available to trigger image refresh for single version devices.
- If your device is a standard version, after pre-storing the image to the corresponding image area of the device, you can use the `33 refresh` command to trigger the pre-stored image to be refreshed.

Field	Type	Required	Value	Remark
opcode	number	Yes	1~UINT32_MAX	Use a new opcode every time you issue a <code>image</code> command to match requests and results. This <code>opcode</code> will be returned in the adv packet(See <code>ds</code> packet in Json-Prepared format for more details)

Field	Type	Required	Value	Remark
img_id	number	Yes	1~UINT32_MAX	Use a new image id every time you issue a diff <code>image</code> to identify image information on the screen. This <code>img_id</code> will be returned in the adv packet(See <code>ds</code> packet in Json-Prepared format for more details)
images	array	Yes	/	image info array
images.data	string	Yes	Http[s] link; Or base64-encoded string of image data	1. The http[s] link is used to download the image data; Or the base64-encoded string of the image data. 2. Do not encode the image data in http[s] link using base64 or any other format. 3. The image format content is not the typical jpg/png format seen on computer desktops. Additional documentation is required to provide details about this specific image format. Please consult with us for further information.
images.screen	array	Yes	A or B	Used to apply the <code>images.data</code> to the screens indicated by <code>images.screen</code> . When there are two screens of the label, A indicates the A side screen of the label, and B indicates the B side of the screen; when the label has only one side, the value can only be A.

Field	Type	Required	Value	Remark
images.compress	string	Yes	NONE or RLE	1. It is used to indicate which compression algorithm is used for the <code>images.data</code>, <code>NONE</code> means no compression algorithm is used, <code>RLE</code> means RLE algorithm 2. While RLE compression can reduce the size of an image and, to some extent, improve the efficiency of bulk image processing, it's important to note that our RLE algorithm is not a standard one; it has undergone special processing. If you have specific requirements, please feel free to consult with us.

Field	Type	Required	Value	Remark
images.region	number	No	0 ~ 7	0. Specifies which region the image will be stored in. 1. Introduced since v3.5.1. If you are using a previous version or do not provide this field, the gateway will default to storing the image starting from region 0. For devices with a single screen, the image will be stored in region 0. For devices with two screens, the first image will be stored in region 0 and the second image will be stored in region 1. 2. The protocol supports storing images in regions 0 to 7, a total of eight images. However, the actual storage capacity depends on the size of the images and the flash space of the device. If the value provided exceeds the maximum capacity of the device, an error will occur and the image cannot be stored on the device. 3. After pre-storing the image to the device's storage space, when you need to update the image, you can use the <code>33 refresh</code> command.

Field	Type	Required	Value	Remark
images.refresh	boolean	No	true/false	0. Specifies whether the device should immediately update the image to the corresponding screen after receiving it. 1. Introduced since v3.5.1. If you are using a previous version or do not provide this field, the gateway will default to true.

set_req

```
{
  "action": 2,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "opcode": 45234,
    "single": true,
    "img_id": 21433,
    "images": [
      {
        "data": "http[s]://host:port/path_to_download_image_data",
        "data": "base64 format of image data",
        /* Please note that the content of "data" can only be selected from
        two options. */
        "screen": ["A"],
        "compress": "NONE"
      }, {
        "data": "http[s]://host:port/path_to_download_image_data",
        "data": "base64 format of image data",
        /* Please note that the content of "data" can only be selected from
        two options. */
        "screen": ["B"],
        "compress": "NONE"
      }
    ],
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

OR Both A & B side share same image

```

{
  "action": 2,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "opcode": 45234,
    "single": true,
    "img_id": 21433,
    "images": [
      {
        "data": "http[s]://host:port/path_to_download_image_data",
        "data": "base64 format of image data",
        /* Please note that the content of "data" can only be selected from
two options. */
        "screen": ["A", "B"],
        "compress": "NONE"
      }
    ],
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}

```

set_rsp

```

{
  "action": 3,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!"
        /* 1. The image data has been successfully sent to the device,
but you need to check the device's advertising packet later to determine if the
image has been successful. */
        /* 2. Only when the `img_id`/`opcode` in the adv packet is equal
to the `img_id`/`opcode` you requested, and the error is 0, it means that the
picture is successfully swiped. */
      }
    }
  }
}

```

03 ota

- Support `set` method only.
- The links and Base64-encoded objects for the firmware are in the form of firmware packages with a ".bin" file extension. If you receive a ZIP file, you need to extract it to obtain the firmware package with the ".bin" file extension.
- Sending the OTA firmware package to the device successfully does not necessarily mean that the OTA process has succeeded. You need to check the device's advertising packet version number or use **74 Read Firmware Version** to determine if OTA has been successful.

Field	Type	Required	Value	Remark
ota	string	Yes	1.Http[s] link; Or base64-encoded string of firmware package	The http[s] link is used to download the firmware upgrade package; 2. Or the base64-encoded string of the firmware package with a ".bin" file extension. 3. Do not encode the firmware data in http[s] link with base64 or any other format.

set_req

```
{
  "action": 3,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload":{
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "single": true,
    "ota": "http[s]://host:port/path_to_download_ota_firmware",
    "ota": "base64 format of ota_firmware",
    /* Please note that the content of "ota" can only be selected from two
options. */
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

set_rsp

```
{
  "action": 3,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
```

```

    "payload": {
      "details": {
        "ac1122334455": {
          "code": 2,
          "error": 0,
          "message": "Success to transmit command data to device!"
          /* The OTA firmware package has been successfully sent to the
device, but you need to check the device's advertising packet (or the No.74
action) later to determine if the OTA process has been successful. */
        }
      }
    }
  }
}

```

20 query

- Support `set` method only.
- Do not use this action if your device is not standard-version.
- This action is based on private 2.4GHz radio.
- This action is used to wake up the devices.
- Since the standard-version device usually broadcasts slowly, if you want to receive timely broadcast information from the devices, you can use this action to wake it up and increase its broadcast frequency. This will allow the gateway to receive its broadcast packets more quickly. After the tag is awakened, it will broadcast advertising info for a certain period of time and then returns to sleep mode.
- **Especially during the initial deployment of devices, if you want to collect broadcast packet data in a timely manner, you can use this command.**
- Since this command is intended for querying the state of the device, the device will not change the `opcode` value after receiving the private 2.4GHz radio command. Therefore, you only need to check the `error code 3` to determine success.
- It is recommended to use the `suppress_stage2` option to suppress the stage 2 result if you don't care about the success or failure result.

set_req

```

{
  "action": 20,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "suppress_stage2": true,
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}

```

set_rsp

```
{
  "action": 26,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 3,
        "message": "Success to wake up the device!",
        "error": 0
      }
    }
  }
}
```

26 shutdown

- Support `set` method only.

set_req

```
{
  "action": 26,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

set_rsp

```
{
  "action": 26,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}
```

```
}  
}  
}
```

33 refresh

- Support `set` method only.
- Do not use this action if your device is not standard-version.
- This action is based on private 2.4GHz radio.
- This action is used to refresh specific image region.
- When the device receives the command 33, if the update is successful, the corresponding `image id` for the picture will be updated on the `img_id` field of the advertising packet with the json-prepared ds type. So, if you want to determine whether the device has performed the update, when using the `02 image` command, you should keep a record of your `image id`. This way, if the `img_id` in the advertising packet matches your value, it indicates that the image update was successful.

Field	Type	Required	Value	Remark
region_a	number	No	0 ~ 7	1. Specify which image region on screen A needs to be refreshed 2. If you don't want to refresh this screen, you can not provide this field 3. The image region number you specify must be pre-stored on the device using the <code>02 image</code> command, otherwise the device will return error code 19.
region_b	number	No	0 ~ 7	1. Specify which image region on screen B needs to be refreshed 2. If the device has only one screen or you don't want to refresh this screen, you can not provide this field 3. The image region number you specify must be pre-stored on the device using the <code>02 image</code> command, otherwise the device will return error code 19.

set_req

```
{  
  "action": 33,  
  "version": 1,  
  "method": "set_req",  
  "req_id": 123,  
  "payload": {  
    "region_a": 0,  
    // "region_b": 0,  
  }  
}
```

```

    "details": {
        "ac1122334455": {},
        "ac1122334456": {}
    }
}

```

set_rsp

```

{
    "action": 26,
    "version": 1,
    "method": "set_rsp",
    "req_id": 123,
    "payload": {
        "details": {
            "ac1122334455": {
                "code": 3,
                "message": "Success to wake up the device!",
                "error": 0
            }
        }
    }
}

```

36 slot

- Support `set` and `get` method.
- Used to configure advertising frames.

field decriptions

common

The table below is common to all [frame types](#).

Field	Type	Required	Value	Remark
slot	array	Required in <code>set_req</code>	/	/
slot_number	number	Required in <code>set_req</code>	0~3	slot number
frame_type	string	Required in <code>set_req</code>	Use string as <code>Frame Type</code>	See Frame Type for details
adv_interval	number	Required in <code>set_req</code>	100~UINT32_MAX	Advertising interval Unit is ms

Field	Type	Required	Value	Remark
adv_by_trigger	boolean	Required in <code>set_req</code>	/	<code>false</code> means always advertising, this means that the device broadcasts advertising frames continuously, but only broadcasts them using the triggered parameters when the trigger event occurs. <code>true</code> means advertising by trigger, this means that the device does not broadcast advertising frames continuously, but only broadcasts them using the triggered parameters when the trigger event occurs.

frame_type

String	Frame Type
FFFF	None
A0FF	Apple ibeacon
00FF	Eddystone UID
10FF	Eddystone URL
20FF	Eddystone TLM
CA00	Minew Device info
CA02	Minew String
CA03	Minew Combination

special for ibeacon

Field	Type	Required	Value	Remark
rss_i_at_xm	number	Required in <code>set_req</code>	-127~0	rss_i at 1m
uuid	string	Required in <code>set_req</code>	/	16-byte in hex string
major	number	Required in <code>set_req</code>	0~UINT16_MAX	/

Field	Type	Required	Value	Remark
minor	number	Required in <code>set_req</code>	0~UINT16_MAX	/

special for uid

Field	Type	Required	Value	Remark
rss_i_at_xm	number	Required in <code>set_req</code>	-127~0	rss_i at 0m
namespace	string	Required in <code>set_req</code>	/	10-byte in hex string
instance	number	Required in <code>set_req</code>	/	6-byte in hex string

special for url

Field	Type	Required	Value	Remark
rss_i_at_xm	number	Required in <code>set_req</code>	-127~0	rss_i at 0m
url	string	Required in <code>set_req</code>	/	http[s] link string

special for string

Field	Type	Required	Value	Remark
string	string	Required in <code>set_req</code>	max length is 18	ascii string

get_req

```
{
  "action": 36,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {}
    }
  }
}
```

get_rsp

```
{
  "action" : 36,
  "req_id" : 123,
  "method" : "get_rsp",
  "payload" : {
    "details" : {
      "ac1122334455" : {
        "code" : 2,
        "message" : "Success to transmit command data to device!",
        "error": 0,
        "slot" : [ {
          "slot_number" : 0,
          "frame_type" : "A0FF",
          "adv_interval" : 500,
          "txpower" : 0,
          "adv_by_trigger" : false,
          "uuid" : "11111111111111111111111111111111",
          "major" : 1,
          "minor" : 1,
          "rssi_at_xm" : -10
        }, {
          "slot_number" : 1,
          "frame_type" : "CA00",
          "adv_interval" : 500,
          "txpower" : 0,
          "adv_by_trigger" : false
        }, {
          "slot_number" : 2,
          "frame_type" : "CA03",
          "adv_interval" : 500,
          "txpower" : 0,
          "adv_by_trigger" : false
        }, {
          "slot_number" : 3,
          "frame_type" : "FFFF",
          "adv_interval" : 0,
          "txpower" : 0,
          "adv_by_trigger" : false
        } ]
      }
    }
  }
}
```

set_req

```
/* Note: In this example, only one channel is set while the other channels retain
their original values. */
{
  "action": 36,
  "version": 1,
```

```

"method": "set_req",
"req_id": 123,
"payload": {
  "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
  "details": {
    "ac1122334455": {
      "slot": [
        {
          "slot_number": 1,
          "frame_type": "A0FF",
          "adv_interval": 100,
          "txpower": 4,
          "adv_by_trigger": false,
          "uuid": "a11b111111111111111111111111cf",
          "major": 20,
          "minor": 10,
          "rssi_at_xm": -69
        }
      ]
    }
  }
}

```

set_rsp

```

{
  "action": 36,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}

```

37 led via radio

- Used to control led light.
- This action is triggered via private 2.4GHz radio channel.
- The parameters are the same as the `42 led via BLE` action, except for the action number.
- Do not use this action if your device is not standard-version.
- Faster than the `42 led via BLE` action.

- You can determine whether the operation is successful by combining the `opcode` and `error` fields in the advertising packet. Alternatively, if a 100% success rate is not required, you can determine success solely based on error code 3 in the second stage. Since this command is triggered by the 2.4GHz radio, if the result from the second stage reply is successful, it is definitely a success. However, if the second stage reply fails, it does not necessarily indicate a failure. Please navigate to [FAQ Q2](#) session for more detailed info.
- It is recommended to use the `suppress_stage2` option to suppress the stage 2 result if you don't care about the success or failure result.

set_req

```
/* The duration of the LED light is 20 cycles, and the length of one cycle is
300ms + 300ms = 600ms. Therefore, the total duration of the LED light is 20 *
600ms = 12000ms. */
{
  "action": 37,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "opcode": 12345678,
    /* opcode: 1. Each time it is issued, it should not be the same as
previous.
                2. If you do not provide this field, the gateway will use the
timestamp in seconds as the opcode. */
    "color": 3,
    "cycles": 20,
    "light_on": 300,
    "light_off": 300,
    "brightness": 50,
    "details": {
      "ac1122334455": {}
    }
  }
}
```

set_rsp

```
{
  "action": 42,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 3,
        "message": "Success to wake up the device!",
        "error": 0
      }
    }
  }
}
```

```
}  
}
```

42 led via BLE

- Support `set` method only.
- Used to control led light.
- This action is connected via BLE channel.
- This action supports both single and standard versions of BLE devices. If your device is the standard version, it is recommended to use `37 led via radio`.
- Do not forget to add `single` field to the json data if your device is standard-version.
- You can determine whether the operation is successful by combining the `opcode` and `error` fields in the advertising packet.

Field	Type	Required	Value	Remark
color	number	YES	0~7	0 indicates turning off the LED light. See Color Table for more details.
cycles	number	YES	1~65535	Total number of cycles to operate, where the duration of one cycle is equal to the sum of <code>light_on</code> and <code>light_off</code> .
light_on	number	YES	1~65535 (ms)	Duration of LED on-time within a single cycle
light_off	number	YES	1~65535 (ms)	Duration of LED off-time within a single cycle
brightness	number	YES	1~100	Brightness of LED

set_req

```
/* The duration of the LED light is 20 cycles, and the length of one cycle is  
300ms + 300ms = 600ms. Therefore, the total duration of the LED light is 20 *  
600ms = 12000ms. */  
{  
  "action": 42,  
  "version": 1,  
  "method": "set_req",  
  "req_id": 123,  
  "payload": {  
    "key": "1234567890123456", /* This is not the real key of the device,  
please consult us to obtain the real key. */  
    "opcode": 12345678,  
    /* opcode: 1. Each time it is issued, it should not be the same as  
previous.  
2. If you do not provide this field, the gateway will use the  
timestamp in seconds as the opcode. */  
    "single": true,
```

```

        "color": 3,
        "cycles": 20,
        "light_on": 300,
        "light_off": 300,
        "brightness": 50,
        "details": {
            "ac1122334455": {}
        }
    }
}

```

set_rsp

```

{
    "action": 42,
    "version": 1,
    "method": "set_rsp",
    "req_id": 123,
    "payload": {
        "details": {
            "ac1122334455": {
                "code": 2,
                "message": "Success to transmit command data to device!",
                "error": 0
            }
        }
    }
}

```

color-table

Value	Color
0	Off
1	Blue
2	Green
3	Red
4	Yellow
5	White
6	Magenta
7	Cyan

44 factoryreset

- Support `set` method only.

set_req

```
{
  "action": 44,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

set_rsp

```
{
  "action": 44,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}
```

52 trigger

- Support `set` and `get` method.
- Used to configure trigger channel.

Field	Type	Required	Value	Remark
trigger	array	Required in <code>set_req</code>	/	/ Max trigger channel number is 6
slot	number	Required in <code>set_req</code>	0~3	slot number
source	number	Required in <code>set_req</code>	1~255	See Tigger Source for details

Field	Type	Required	Value	Remark
interval	number	Required in set_req	100~UINT32_MAX	Unit is ms,advertising interval after triggering
duration	number	Required in set_req	100~UINT32_MAX	Unit is ms, broadcast duration after triggering
txpower	number	Required in set_req	-40, -20, -16, -12, -8, -4, 0, 4	advertising txpower when trigger occurs

trigger_source

Value	Trigger Source
0	None
1	Single click
2	Double click
3	Tripple click
4	Long pressed
5	High temperature detected
6	Low temperature detected
7	High humidity detected
8	Low humidity detected
9	ACC trigger
10	Button pressed (for tamper-proof beacon)
11	Button released (for tamper-proof beacon)
12~255	For future use

get_req

```
{
  "action": 52,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

get_rsp

```
{
  "action": 52,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {
        "trigger": [
          {
            "slot": 0,
            "source": 2,
            "interval": 100,
            "duration": 10000,
            "txpower": 4
          },
          {
            "slot": 2,
            "source": 5,
            "interval": 100,
            "duration": 10000,
            "txpower": 4
          }
        ]
      }
    }
  }
}
```

set_req

```
{
  "action": 52,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "trigger": [
      {
        "slot": 0,
        "source": 2,
        "interval": 100,
        "duration": 10000,
        "txpower": 4
      },
      {
        "slot": 2,
        "source": 5,
        "interval": 100,
        "duration": 10000,
        "txpower": 4
      }
    ],
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

set_rsp

```
{
  "action": 52,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}
```

54 HT data

- Support `get` method only.
- used to get the historical data (temperature & humidity) saved by the device.

Field	Type	Included	Value	Remark
time_start	number	Included in <code>get_req</code>	The value is a four-byte integer ranging from 0 to 4294967295	The start timestamp refers to the Uinx timestamp.
time_stop	number	Included in <code>get_req</code>	The value is a four-byte integer ranging from 0 to 4294967295	The end timestamp refers to the Uinx timestamp.
items	array	Included in <code>get_rsp</code>	/	/
timestamp	number	Included in <code>get_rsp</code>	The value is a four-byte integer ranging from 0 to 4294967295	Uinx timestamp.
temperature	number	Included in <code>get_rsp</code>	/	temperature
humidity	number	Included in <code>get_rsp</code>	/	humidity

get_req

```
{
  "action": 54,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "details": {
      "c20303002293": {
        "time_start": 1708247719,
        "time_stop": 1708247720
      }
    }
  }
}
```

get_rsp

```
{
  "action": 54,
  "req_id": 123,
  "version": 1,
  "method": "get_rsp",
  "payload": {
    "details": {
      "c20303002293": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!",
        "items": [{
          "timestamp": 1708662833,
          "temperature": 24.91,
          "humidity": 57.03
        }, {
          "timestamp": 1708662863,
          "temperature": 25.28,
          "humidity": 56.31
        }]
      }
    }
  }
}
```

56 buzzer

- Support `set` method only.

Field	Type	Required	Value	Remark
cycles	number	Required in <code>set_req</code>	A positive integer of four bytes, ranging from 0 to 4294967295	1. The number of cycles that the buzzer will work. A value of 0 means the buzzer is turned off. 2. The length of one cycle is equal to the sum of "work" time and "idle" time.
work	number	Required in <code>set_req</code>	20~65535 milliseconds	The duration of the buzzer's sound in a single cycle.
idle	number	Required in <code>set_req</code>	20~65535 milliseconds	The duration of the buzzer's silence in a single cycle.

Field	Type	Required	Value	Remark
level	number	Required in <code>set_req</code>	0, 1, 2, 3, 4, 5	1. The frequency level of the buzzer. The value of the instruction must not exceed the maximum level supported by the device, otherwise, an error code 20 will be reported. 2. For example, if the device supports 5 levels, the levels are numbered from 1 to 5. 0 represents the default level. 3. Not all devices support 5 levels.

set_req

```

/* The duration of the buzzer's sound is 20 cycles, and the length of one cycle
is 300ms + 300ms = 600ms. Therefore, the total duration of the buzzer's sound is
20 * 600ms = 12000ms. */
{
  "action": 56,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "cycles": 20,
    "work": 300,
    "idle": 300,
    "level": 0,
    "details": {
      "ac1122334455": {}
    }
  }
}

```

set_rsp

```

{
  "action": 56,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}

```

```
}  
}  
}
```

57 scan

- Support `set` and `get` method.
- Regarding turning on and off BLE scanning, please see **67 switches**.

Field	Type	Required	Value	Remark
active	boolean	Required in <code>set_req</code>	/	1. Whether to enable active scanning: true for active scanning and false for passive scanning; 2. Enabling active scanning will retrieve scan response packets.
interval	number	Required in <code>set_req</code>	5~10000	Scan interval, range: 5-10000, unit: ms; can only be set to multiples of 5.
windows	number	Required in <code>set_req</code>	5~10000	Scan window, range: 5-10000, unit: ms; can only be set to multiples of 5. Note: windows value must be less than or equal to interval value.
timeout	number	Required in <code>set_req</code>	0~655350	Scan timeout, range: 0-655350, unit: ms; can only be set to multiples of 10 and must be a multiple of the interval parameter. 0 means no timeout.

get_req

```
{  
  "action": 57,  
  "version": 1,  
  "method": "get_req",  
  "req_id": 123,  
  "payload": {  
    "key": "1234567890123456", /* This is not the real key of the device,  
    please consult us to obtain the real key. */  
    "details": {  
      "ac1122334455": {}  
    }  
  }  
}
```

get_rsp

```
{
  "action": 57,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0,
        "active": false,
        "interval": 100,
        "windows": 100,
        "timeout": 2000
      }
    }
  }
}
```

set_req

```
{
  "action": 57,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "active": false,
    "interval": 100,
    "windows": 100,
    "timeout": 2000,
    "details": {
      "ac1122334455": {}
    }
  }
}
```

set_rsp

```
{
  "action": 57,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
```

```

        "message": "Success to transmit command data to device!",
        "error": 0
    }
}
}
}

```

58 reboot

- Support `set` method only.

set_req

```

{
  "action": 58,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}

```

set_rsp

```

{
  "action": 58,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}

```

62 timestamp

- Support `set` and `get` method.
- used to update the timestamp of the device.

Field	Type	Required	Value	Remark
timestamp	number	Required in <code>set_req</code>	The value is a four-byte integer ranging from 0 to 4294967295	Unix timestamp (also known as Epoch time) is the number of seconds that have elapsed since January 1, 1970, 00:00:00 UTC (Coordinated Universal Time). It is not based on a specific time zone but rather on international standard time, which is UTC (Coordinated Universal Time). When the value of the timestamp you send is 0, the gateway will send the system's Unix timestamp as a parameter to the device.

get_req

```
{
  "action": 62,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "details": {
      "c20303002293": {}
    }
  }
}
```

get_rsp

```
{
  "action": 62,
  "req_id": 123,
  "version": 1,
  "method": "get_rsp",
  "payload": {
    "details": {
      "c20303002293": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!",
        "timestamp": 1708662833
      }
    }
  }
}
```

set_req

```
{
  "action": 62,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "timestamp": 1708662833,
    "details": {
      "c20303002293": {}
    }
  }
}
```

set_rsp

```
{
  "action": 62,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "c20303002293": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}
```

63 historical data

- Support `get` method only.
- used to get the historical data (Universal Command) saved by the device.

Field	Type	Included	Value	Remark
rule	number	Included in <code>get_req</code>	0 or 1	rule
type	number	Included in <code>get_req</code>	0~5	type
time_start	number	If rule=1, the value is included in <code>get_req</code> .	The value is a four-byte integer ranging from 0 to 4294967295	The start timestamp refers to the Uinx timestamp.

Field	Type	Included	Value	Remark
time_stop	number	If rule=1, the value is included in <code>get_req</code> .	The value is a four-byte integer ranging from 0 to 4294967295	The end timestamp refers to the Uinx timestamp.
timestamp	number	Included in <code>get_rsp</code>	The value is a four-byte integer ranging from 0 to 4294967295	Uinx timestamp.
temperature	number	If type=2 or type=3, the value is included in <code>get_rsp</code> .	/	/
humidity	number	If type=2 , the value is included in <code>get_rsp</code> .	/	/
unlocked	number	If type=5 , the value is included in <code>get_rsp</code> .	0 or 1	0: The door status sensor is locked; 1: The door status sensor is unlocked.
uninstalled	number	If type=5 , the value is included in <code>get_rsp</code> .	0 or 1	0: The anti-disassembly switch is removed; 1: The anti-disassembly switch is in normal state.

rule

Data	Detail	
0	get all data	
1	get data by time period	

type

Data	Type	Remark
0	Reserved	/
1	GNSS	/
2	Temperature & Humidity	completed

Data	Type	Remark
3	Temperature Only	completed
4	Advertising	/
5	Door Magnet & Tamper Status	completed

get_req

- If the `rule` field is 1, the `time_start` and `time_stop` fields are required.

```
{
  "action": 63,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "c20303002293": {
        "rule": 1,
        "type": 3, // 2,3,4,5
        "time_start":1708247719,
        "time_stop":1708247720
      }
    }
  }
}
```

- If the `rule` field is 0, the `time_start` and `time_stop` fields are not required.

```
{
  "action": 63,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "c20303002293": {
        "rule": 0,
        "type": 3 // 2,3,4,5
      }
    }
  }
}
```

get_rsp

- The `type` field is 2.

```
{
  "action": 63,
  "req_id": 123,
  "version": 1,
  "method": "get_rsp",
  "payload": {
    "details": {
      "c20303002293": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!",
        "items": [{
          "timestamp": 1708662833,
          "temperature": 24.91,
          "humidity": 44.53
        }, {
          "timestamp": 1708662863,
          "temperature": 25.28,
          "humidity": 44.54
        }]
      }
    }
  }
}
```

- The `type` field is 3.

```
{
  "action": 63,
  "req_id": 123,
  "version": 1,
  "method": "get_rsp",
  "payload": {
    "details": {
      "c20303002293": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!",
        "items": [{
          "timestamp": 1708662833,
          "temperature": 24.91
        }, {
          "timestamp": 1708662863,
          "temperature": 25.28
        }]
      }
    }
  }
}
```

- The `type` field is 5.

```
{
  "action": 63,
  "req_id": 123,
  "version": 1,
  "method": "get_rsp",
  "payload": {
    "details": {
      "c20303002293": {
        "code": 2,
        "error": 0,
        "message": "Success to transmit command data to device!",
        "items": [{
          "timestamp": 1708662833,
          "unlocked" : 1
        }, {
          "timestamp": 1708662863,
          "uninstalled" : 1
        }]
      }
    }
  }
}
```

67 switches

- Support `set` and `get` method;

Field	Type	Required	Value	Remark
switches	array	Required in <code>set_req</code>	/	/
option	number	Required in <code>set_req</code>	option	Represents the option being controlled; Not all devices support all options, use <code>get_req</code> method to obtain the supported options.
value	number	Required in <code>set_req</code>	0 or 1	Used to switch configurations.

get_req

```
{
  "action": 67,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {}
    }
  }
}
```

get_rsp

```
{
  "action": 67,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0,
        "switches": [
          {
            "option": 3,
            "value": 1
          },
          {
            "option": 1,
            "value": 1
          }
        ]
      }
    }
  }
}
```

set_req

```
{
  "action": 67,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
```

```

    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "switches": [
        {
            "option": 3,
            "value": 0
        }
    ],
    "details": {
        "ac1122334455": {}
    }
}

```

set_rsp

```

{
    "action": 67,
    "version": 1,
    "method": "set_rsp",
    "req_id": 123,
    "payload": {
        "details": {
            "ac1122334455": {
                "code": 2,
                "message": "Success to transmit command data to device!",
                "error": 0
            }
        }
    }
}

```

option

Type	Detail
0	Magnetron detection
1	Bluetooth broadcasting
2	Screen display temperature unit: Fahrenheit/Celsius When set to 0, the temperature on the screen is displayed in Celsius, and 1 is used for Fahrenheit.
3	Bluetooth scanning 0 means disabling BLE scanning, while 1 means enabling it
4 ~ 127	Reserved

74 version

- Support `get` method only;

Field	Type	Included	Value	Remark
firmware	array	Included in <code>get_rsp</code>	/	/
type	number	Included in <code>get_rsp</code>	firmware_type	Firmware type; Some products have more than one firmware.
version	string	Included in <code>get_rsp</code>	vx.x.x	Version number of the corresponding firmware type.

get_req

```
{
  "action": 74,
  "version": 1,
  "method": "get_req",
  "req_id": 123,
  "payload": {
    "key": "1234567890123456", /* This is not the real key of the device,
    please consult us to obtain the real key. */
    "details": {
      "ac1122334455": {}
    }
  }
}
```

get_rsp

```
{
  "action": 74,
  "version": 1,
  "method": "get_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0,
        "firmware": [
          {
            "type": 0,
            "version": "v3.2.0"
          }
        ]
      }
    }
  }
}
```

```
}  
}
```

firmware_type

Data	Type
0	Application Core
1	Net Core
2	Application of Cellular Module
3	Modem of Cellular Module

75 operating status

- Support `get` method only;

Field	Type	Included	Value	Remark
battery	number	Included in <code>get_rsp</code>	0~100	Battery Level
time_since_boot	number	Included in <code>get_rsp</code>	/	Time Since Boot in Seconds

get_req

```
{  
  "action": 75,  
  "version": 1,  
  "method": "get_req",  
  "req_id": 123,  
  "payload": {  
    "key": "1234567890123456", /* This is not the real key of the device,  
    please consult us to obtain the real key. */  
    "details": {  
      "ac1122334455": {}  
    }  
  }  
}
```

get_rsp

```
{  
  "action": 75,  
  "version": 1,  
  "method": "get_rsp",  
  "req_id": 123,  
  "payload": {
```

```

    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0,
        "battery": 100,
        "time_since_boot": 12000
      }
    }
  }
}

```

101 Multi-Screen Image Region Refresh via Radio

- Only supports the `set` method.
- Do not use this action if your device is not a standard version.
- This action is triggered through a private 2.4GHz radio channel.
- This action is an upgraded version of command 33, which can support refreshing specified image regions on up to five screens (A, B, C, D, E) simultaneously.
- If you want to refresh image regions via BLE channel, use command 102.
- When the device receives command 101, if the update is successful, the `img_id` field in the advertising packet will be updated to the corresponding image ID.

Field	Type	Required	Value	Remark
region_a	number	No	0 ~ 31	1. Specifies the image region on screen A to be refreshed 2. If you don't want to refresh this screen, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.
region_b	number	No	0 ~ 31	1. Specifies the image region on screen B to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.

Field	Type	Required	Value	Remark
region_c	number	No	0 ~ 31	1. Specifies the image region on screen C to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.
region_d	number	No	0 ~ 31	1. Specifies the image region on screen D to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.
region_e	number	No	0 ~ 31	1. Specifies the image region on screen E to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.

set_req

```
{
  "action": 101,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
    "region_a": 0, /* Refresh the image previously stored in region 0 to
screen a */
    "region_c": 1,
    "region_e": 2,
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

set_rsp

```
{
  "action": 101,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 3,
        "message": "Success to wake up device!",
        "error": 0
      }
    }
  }
}
```

102 Multi-Screen Image Region Refresh via BLE

- Only supports the `set` method.
- This action is the BLE version of command 101, connecting through the BLE channel.
- This action is used to refresh specified image regions on up to five screens (A, B, C, D, E).
- If you want to refresh image regions via private 2.4GHz radio channel, use command 101.
- When the device receives command 102, if the update is successful, the `img_id` field in the advertising packet will be updated to the corresponding image ID.

Field	Type	Required	Value	Remark
region_a	number	No	0 ~ 31	1. Specifies the image region on screen A to be refreshed 2. If you don't want to refresh this screen, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.
region_b	number	No	0 ~ 31	1. Specifies the image region on screen B to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.

Field	Type	Required	Value	Remark
region_c	number	No	0 ~ 31	1. Specifies the image region on screen C to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.
region_d	number	No	0 ~ 31	1. Specifies the image region on screen D to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.
region_e	number	No	0 ~ 31	1. Specifies the image region on screen E to be refreshed 2. If the device doesn't have this screen or you don't want to refresh it, you can omit this field 3. The image region number you specify must be pre-stored on the device using command 103, otherwise the device will return error code 19.

set_req

```
{
  "action": 102,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload":{
    "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
    "region_a": 0, /* Refresh the image previously stored in region 0 to
screen a */
    "region_c": 1,
    "region_e": 2,
    "details": {
      "ac1122334455": {},
      "ac1122334456": {}
    }
  }
}
```

set_rsp

```
{
  "action": 102,
  "version": 1,
  "method": "set_rsp",
  "req_id": 123,
  "payload": {
    "details": {
      "ac1122334455": {
        "code": 2,
        "message": "Success to transmit command data to device!",
        "error": 0
      }
    }
  }
}
```

103 Multi-Panel Image Download & Screen Refresh

- Only supports the `set` method.
- Used to transmit image data to multi-screen devices (supporting up to five screens: A, B, C, D, E).
- This command is an upgraded version of command 02, supporting more screens and a larger range of image region numbers.
- After successfully sending image data to the device, you can use command 101 or 102 to refresh the pre-stored images.

Field	Type	Required	Value	Remark
opcode	number	Yes	1~UINT32_MAX	Use a new opcode each time you issue an <code>image</code> command to match requests and results. This <code>opcode</code> will be returned in the advertising packet.
images	array	Yes	/	Array of image information
images.data	string	Yes	Http[s] link; or base64 encoded string of image data	1. Http[s] link to download image data; or base64 encoded string of image data. 2. Do not encode image data in HTTP[s] link with base64 or any other format.

Field	Type	Required	Value	Remark
images.screen	array	Yes	[A, B, C, D, E]	Used to specify which screens the <code>images.data</code> applies to. The protocol supports specifying multiple screens at once (e.g., ["A", "C", "E"]), allowing a single image data to be refreshed to multiple screens, but actual support depends on the device implementation. This design is mainly for compatibility with command 02's style, and some devices may only support refreshing a single screen.
images.compress	string	Yes	NONE or RLE	Used to indicate which compression algorithm is used for <code>images.data</code> , NONE means no compression algorithm is used, RLE means RLE algorithm is used.

Field	Type	Required	Value	Remark
images.region	number	Yes	0 ~ 31	Specifies which region the image will be stored in. The protocol supports region numbers 0-31 (32 image regions total), but the actual maximum supported region number depends on the device's storage space and the corresponding screen size. The total area of regions is related to the size of the EEPROM supported by the device. The number of regions equals the total area of the device's supported regions divided by the fixed number of pages for the device's image (4KByte per page).
images.refresh	boolean	Yes	true/false	Specifies whether the device should immediately update the image to the corresponding screen after receiving it. If set to false, you can later use command 101 or 102 to refresh the image.

Field	Type	Required	Value	Remark
images.img_id	number	Yes	1~UINT32_MAX	Use a new image ID each time you issue a different <code>image</code> to identify the image information on the screen. This <code>img_id</code> will be returned in the advertising packet. Note that this field's position differs from the <code>img_id</code> in command <code>02 image</code>. If multiple screens are refreshed simultaneously, the <code>img_id</code> field in the json-prepared ds type will only be set to the <code>img_id</code> of the last picture. You can determine whether the image was successfully refreshed by checking if the <code>img_id</code> in the advertising packet equals the last image id you sent, because if there is an image that fails to refresh in the middle, the <code>img_id</code> in the broadcast packet will only be the last successfully refreshed one. Therefore, if the last image ID you sent differs from the <code>img_id</code> in the broadcast packet, it indicates that there was a failure.

set_req

```
{
  "action": 103,
  "version": 1,
  "method": "set_req",
  "req_id": 123,
  "payload": {
```

```

        "key": "1234567890123456", /* This is not the real key of the device,
please consult us to obtain the real key. */
        "opcode": 45234,
        "images": [
            {
                "data": "http[s]://host:port/path_to_download_image_data",
                "data": "base64 format of image data",
                /* Note that the content of "data" can only be chosen from two
options. */
                "screen": ["A", "C", "E"],
                "compress": "NONE",
                "region": 0,
                "refresh": true,
                "img_id": 21433
            }, {
                "data": "http[s]://host:port/path_to_download_image_data",
                "data": "base64 format of image data",
                /* Note that the content of "data" can only be chosen from two
options. */
                "screen": ["B", "D"],
                "compress": "NONE",
                "region": 1,
                "refresh": false,
                "img_id": 21434
            },
            {
                ....
            }
        ],
        "details": {
            "ac1122334455": {},
            "ac1122334456": {}
        }
    }
}

```

set_rsp

```

{
    "action": 103,
    "version": 1,
    "method": "set_rsp",
    "req_id": 123,
    "payload": {
        "details": {
            "ac1122334455": {
                "code": 2,
                "error": 0,
                "message": "Success to transmit command data to device!"
                /* 1. The image data has been successfully sent to the device,
but you need to check the device's advertising packet later to determine if the
image was successful. */
                /* 2. The picture is only successfully refreshed when the
`img_id`/`opcode` in the advertising packet equals the `img_id`/`opcode` you
requested, and the error is 0. */
            }
        }
    }
}

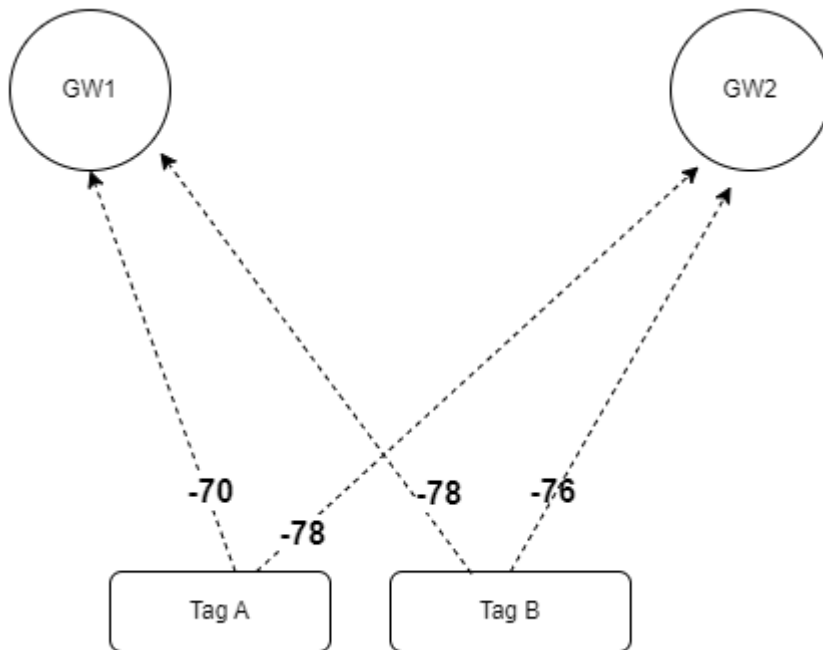
```

```
}  
  }  
}  
}
```

FAQ

Q1

Q: As the following figure show, how to send commands to multiple gateways in a multi-gateway situation?



A:

- Due to the limited BLE connection distance of the gateway, it is possible that multiple gateways may be deployed in one store. When multiple gateways exist, how the platform can issue commands to the corresponding gateways is a worthwhile discussion. You can choose different solutions based on your device type.
- Solution 1:
 - To determine the distance between the BLE devices and the gateway based on the RSSI value, the larger the RSSI value, the shorter the distance. You can send the command to the gateway with a higher signal value to connect and control the BLE devices.
 - The RSSI value is reported together with the information in the advertising packets. For more details, please refer to the documentation on [Json-Prepared](#).
 - During the initial deployment, since the broadcasting of the BLE devices is slow, if you want to receive the RSSI value from the device promptly, you can wake up the device first using the action `20 query`.
 - Due to the longer communication distance and faster efficiency of the 2.4GHz radio instructions compared to BLE, for commands like `20 query` and `37 led via radio` based on the 2.4GHz radio, you can send the command to any gateway. However, if your deployment area is larger than the coverage range of the 2.4GHz radio, you can send the same wake-up command to all gateways, and they will all respond.

- As shown in the figure above, for Tag A, the RSSI value received by GW1 is -70, and the RSSI value received by GW2 is -78. Therefore, you should choose GW1 as the control gateway for Surf A. Similarly, for Tag B, you should choose GW2. **When both GW1 and GW2 receive the same RSSI value for the device, you can choose either gateway.**
- Solution 2:
 - When your BLE device is a standard version, you can send your command to multiple gateways simultaneously, and the gateway's own working mechanism can avoid duplicate operations. However, the gateway's return error code in the second stage may not be accurate, which means that the platform needs to integrate multiple gateways' returned error codes to ultimately confirm whether the operation has failed.
 - Only when all gateways return error codes indicating failure does it indicate that the BLE device operation has failed.
 - If any one of the gateways returns success code, it means that the BLE device operation has succeeded.
 - Therefore, when one of the gateways returns failure, it cannot be immediately determined as a failure. You need to wait for all gateways to return before confirming it as a failure.
 - Note that the `opcode` field parameter must also be issued, and it must be consistent at multiple gateways. The gateway uses `opcode` to avoid duplicate operations.
 - When your BLE device is a single version, sending commands to multiple gateways simultaneously may result in some devices being operated multiple times, **which may lead to increased power consumption.**

Q2

Q: Why is the response inaccurate during the second stage of the `37 led via radio` command or other private 2.4GHz radio-based command?

A: Because the success of the gateway's 2.4GHz radio command relies on the tag's BLE advertising packets, when the tag is at a considerable distance, it might have received the 2.4GHz radio command. However, due to distance-related factors (increased packet loss in BLE broadcast packets at longer distances) and limitations in the gateway's judgment time (the gateway considers the command failed if it doesn't receive the device's BLE advertising packet reply within a certain timeframe), situations may arise where the tag has succeeded, but the gateway incorrectly judges it as a failure. To mitigate this scenario, deploying multiple gateways can be considered. Alternatively, use the `suppress_stage2` option to suppress the results of the second stage, and directly determine success or failure based on the `opcode` and `error` fields in the advertising packet.

Q3

Q: When your system has multiple gateways, if you simultaneously send the same private 2.4GHz radio-based command to all gateways, is it possible for multiple gateways to return success in the second stage at the same time?

A: Yes, there can be a situation where multiple gateways return success in the second stage simultaneously. This occurs because gateways rely on advertising packets to determine success. Multiple gateways may indeed simultaneously return success in the second stage, but it doesn't imply that the tag operation was executed twice. This is due to the control provided by the

`opcode`; as long as the opcode value in the command is the same, there is no need to worry about duplication.

Q4

Q: When the `37 led via radio` command cannot be sent in batches together, how can the speed of sending radio-based commands for `37 led via radio` be increased as much as possible? (This also applies to all private 2.4GHz radio-based commands that require passing `opcode` params.)

A: As is well known, when commanding multiple LED simultaneously through a single message, the speed is quite fast. However, if you split the command for controlling multiple LED into separate messages and send them one by one, the process becomes significantly slower. This is because the gateway employs a merging logic operation, where only messages with the same opcode are combined and sent to the device side simultaneously. If your messages do not carry the `opcode` parameter, the gateway will use the timestamp as the `opcode`. This results in the gateway sending data to the device side one by one, making the process much slower than when sending in batches. Therefore, if you want to increase the speed, you need to perform the following tasks:

- When sending messages, you should include your opcode.
- You should ensure that the opcode remains as consistent as possible for different tag devices. This way, the gateway can merge commands with the same opcode together for simultaneous operation as much as possible.
- At the same time, you also need to ensure that for the same tag device, you should not use the same opcode value consecutively. This is because tag devices always store the last used opcode value, and if the same opcode is used again the second time, the tag will consider it as the same operation. As a result, subsequent operations with the same opcode will no longer take effect.

Q5

Q:

- What is the difference between jengine v1 and v2?
- Why added jengine v2?
- Whether jengine v1 and v2 can be used together after v3.7.0 version?

A:

- Compared with v1, v2 can deliver multiple actions at a time. The `action`, `method` and `payload` fields are put into the new `cmds` array field to deliver multiple actions at a time.
- If v1 needs to issue two commands to the same device in a row, the gateway needs to connect to the device twice, which is inefficient, whereas v2 can merge the data and issue the two commands to the device together, and only connect once. For example, for the MBT02 device, to issue a `42 led via BLE` and a `56 buzzer`, v1 needs to issue two commands while v2 only needs to be merged into `cmds` to issue one command.
- Yes, after v3.7.0 both v1 and v2 can be used, and v2 can also issue a one command